

Progressive disclosure

[Wikipedia states](#): "*Progressive disclosure is an interaction design pattern used to make applications easier to learn and less error-prone. It does so by deferring some advanced or rarely-used features to a secondary screen.*"

In 2006, Jakob Nielsen [formulated](#) this pattern as follows:

"Progressive disclosure defers advanced or rarely used features to a secondary screen, making applications easier to learn and less error-prone."

That is one of the most powerful and underestimated design patterns. The idea is quite simple.

On the one hand, your app needs to provide many features, which is good for users.

On the other hand, users can be intimidated by the amount of information shown at once. If you simply output every possible action of your UI in one place, it will be very confusing for users.

The purpose of the progressive disclosure pattern is to show only key features (as little as possible) at first, but if users indicate that they need more - then give them more.

Consider the following form below: users need to fill in their name, two date fields, and phone number. Those fields are required.

However, if they have any "Special Requirements", they can add them to the text area, which is marked "Optional".

If you used the progressive disclosure pattern, you would have a checkbox asking users first if they have such "Special Requirements", and if they check it, only then does the form appear.

Notice that, in this case, the text area is required - not optional.

The image displays two versions of a 'Book a Room' form side-by-side, illustrating the concept of progressive disclosure.

Form 1 (Left): Titled 'Book a Room', it asks the user to 'Please fill the form below to make a reservation'. It includes fields for 'Your Name', 'Since' (DD/MM), 'Till' (DD/MM), 'Phone Number', and 'Special Requirements (optional)'. The 'Special Requirements' field is a text area with the placeholder 'Anything we need to know in advance'. A blue 'Send' button is at the bottom.

Form 2 (Right): Also titled 'Book a Room', it has the same initial fields as Form 1. However, it includes a checkbox labeled 'I want to specify special requirements' which is checked. Below this checkbox is the 'Special Requirements' text area. A blue arrow points from the checkbox to the text area. A dashed blue arrow points downwards from the text area, and the text 'Not optional anymore' is written below it. A blue 'Send' button is at the bottom.

Even though the example is quite trivial, the form might get bigger. Also, imagine you have the option to pick users up from a certain location- that adds a few additional fields that are still optional (without progressive disclosure).

In the first form, we have 3 additional inputs (special requirements, city, address) which are optional. We display them right away, which might end up looking like a big complex form.

In the second form, we have a couple of checkboxes. Their function is to reveal additional options.

The diagram illustrates a form transformation pattern. On the left is a standard 'Book a Room' form with fields for Name, dates (Since/Till), phone number, optional special requirements, and optional pickup details (City/Address). A 'Send' button is at the bottom. On the right is the same form but with two checkboxes added: 'I want to specify special requirements' and 'I want you to pick me up'. Both are checked. Blue arrows point from these checkboxes to the 'Special Requirements', 'City', and 'Address' fields, indicating they are now required. A dashed blue arrow on the right points downwards, and a label 'Not optional anymore' points to the bottom of the form.

Book a Room
Please fill the form below to make a reservation

Your Name

Since DD/MM Till DD/MM

Phone Number

Special Requirements (optional)

If you want us to pick you up, provide details below

City (optional)

Address (optional)

Send

Book a Room
Please fill the form below to make a reservation

Your Name

Since DD/MM Till DD/MM

Phone Number

☒ I want to specify special requirements

Special Requirements

☒ I want you to pick me up

City

Address

Send

Not optional anymore

This is a very handy pattern that can be used in (but not limited to) such cases as:

- I need delivery (reveals multiple inputs related to addresses)
- Please, call me (reveals phone number input)
- Show advanced options

Data saving

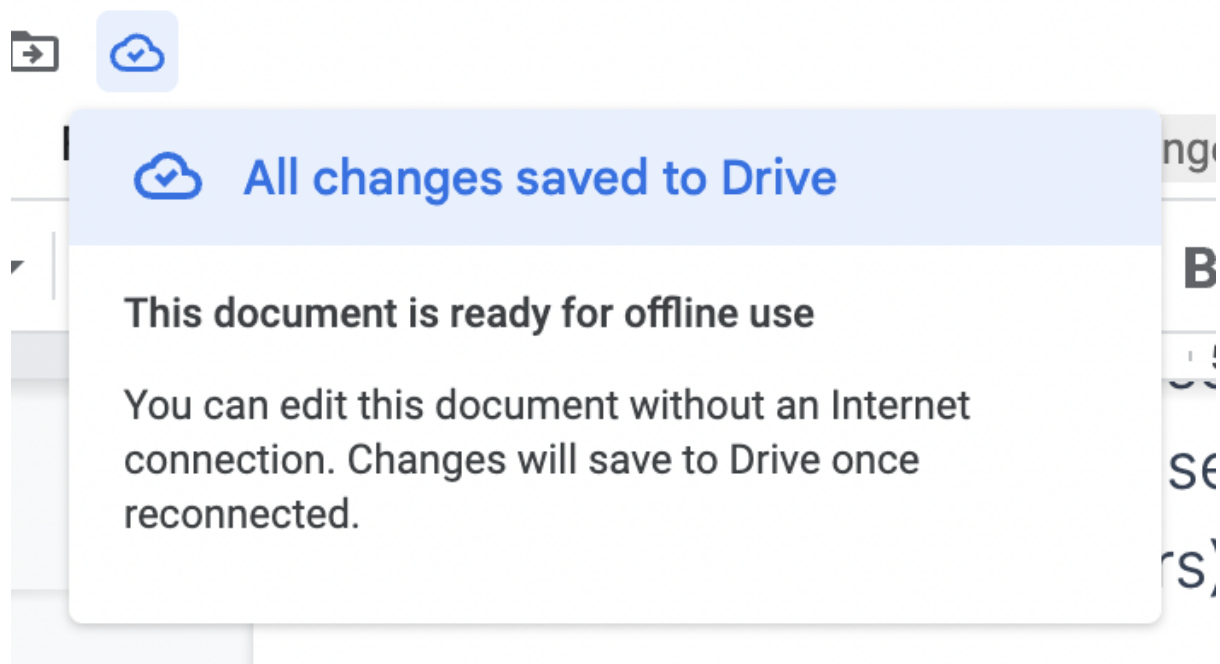
One of the most important principles of UI is that user data is priceless.

Have you ever experienced such a situation: you're writing an article in Word and then suddenly your computer stops working and, as a result, you lose your data simply because you forgot to save it in advance?

Instead of making users save data, do it automatically. Especially in such forms where users have to type a lot of data (comments, articles, code editors).

In most situations, you will need some kind of indicator that the data is saved. For example, right now, as I'm writing these lines in Google docs I can see the "Saving..." label at the top of the page.

When the data is saved, it says "Saved to Drive". Moreover, it gives me some additional details.



Respect Local Differences

If your application is used in multiple countries, then your forms should be modified accordingly for each one.

In the following example, Russia doesn't have a State or Zip code, so there is no point in asking for it.

Instead, you should ask for a "Postal code" that is more suitable, and remove the "State" field completely.

Address settings

Manage your address settings

Country

Russia

Zip Code

17364

City

Moscow

State

Address

Some Street

Address settings

Manage your address settings

Country

Russia

Postal Code

17364

City

Moscow

Address

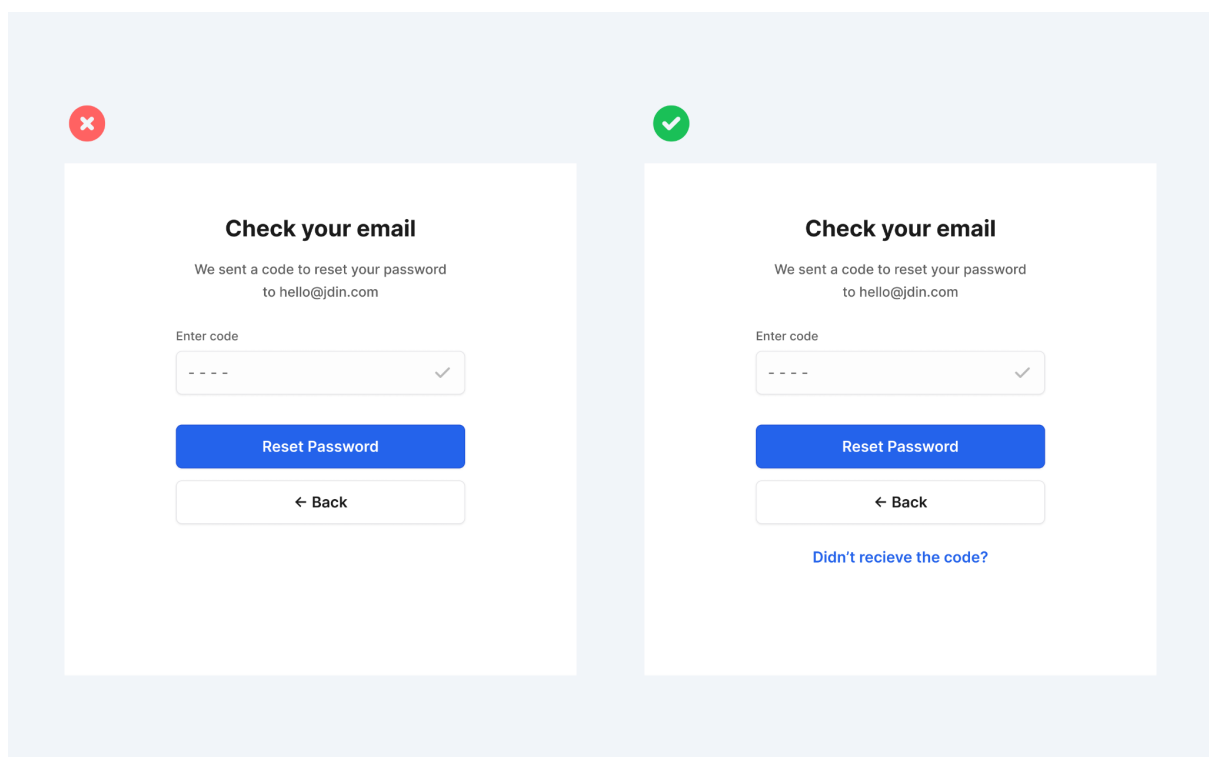
Some Street

Provide further instructions

This is one of the most **crucial** things in UI/UX design.

When something goes wrong, it is not enough to notify users about it. You should also provide instructions on what they can do. Otherwise, they are at a dead-end.

In the example below a small link “Didn’t receive the code” makes a huge difference from the UX point of view.



I found A nice example from a Russian bank that sends SMS for verifying transactions.

Sometimes SMS might not work, and they provided a tooltip with helpful information on what can be done in case you didn't receive an SMS.